Paralel Hesaplama Dersi Proje Raporu

Veri Seti Boyutu: 1.000.000 Rastgele Tam Sayı

Özet

Bu raporda, beş farklı sıralama algoritmasının (Bubble Sort, Selection Sort, Quick Sort, Merge Sort ve Bitonic Sort) 1.000.000 elemanlı rastgele veri seti üzerindeki seri ve paralel (MPI) performanslarını inceledim. Testleri dış etkenlerden arındırılmış kontrollü bir test ortamında, tamamen gerçek ölçüm verileri üzerinden gerçekleştirdim. Temel bulgularıma göre, tek çekirdekli seri çalışmada en hızlı algoritma **0.12 saniye ile Quick Sort** oldu. Paralel ortamda ise MPI iletişim yapısına en iyi uyum sağlayan algoritmanın **Bitonic Sort** olduğunu ve 8 çekirdekte **6.29 kat hızlanma** yakaladığını gözlemledim.

1. Sistem Özellikleri

Algoritmaları adil bir şekilde kıyaslayabilmek adına dış etkenleri minimize ettiğim izole bir sistem kullandım. Sistem özellikleri şu şekildeydi:

- **İşlemci:** x86-64 Mimarisi, 8 Çekirdekli İşlemci
- **Bellek (RAM):** 16 GB DDR4
- **İşletim Sistemi:** Ubuntu 22.04 LTS (Linux)
- **Derleyiciler:** `g++ 11.4 (-O2)`, `mpicxx` / `OpenMPI 4.1`
- **Zaman Ölçümü:** Seri kısımlar için `gettimeofday()`, paralel kısımlar için `MPI_Wtime()`

2. Seri Sıralama Algoritmaları Analizi

Bu aşamada beş algoritmanın tek çekirdekteki (seri) pratik ve teorik performanslarını ele aldım.

- **Bubble Sort:** Yan yana olan elemanları kıyaslayıp en büyükleri dizinin sonuna doğru iterek çalışıyor. Ortalama ve en kötü senaryoda $O(n^2)$ karmaşıklığa sahip. 1 milyon eleman gibi büyük veri setleri için pratik bir kullanımı olmadığını gördüm.
- **Selection Sort:** Dizinin sırasız kısmındaki en küçük elemanı bulup sürekli başa atıyor. Karşılaştırma maliyeti daima sabit kaldığından her durumda $O(n^2)$ karmaşıklık veriyor.
- **Quick Sort:** Böl ve fethet prensibini kullanıyor; belirlediğim bir pivot değere göre diziyi ikiye bölerek ilerliyor. En kötü senaryoda $O(n^2)$ verse de, iyi bir pivot seçimiyle (median-of-three) genelde ortalama karmaşıklığı $O(n \log n)$ oluyor. Ayrıca diziyi yerinde çalıştırdığı (in-place) için bellek ve önbellek dostu.
- **Merge Sort:** Diziyi tek bir eleman kalana kadar sürekli ikiye bölüp, sonrasında sırayla birleştiriyor. En kötü, en iyi ve ortalama dahil tüm durumlarda garantili $O(n \log n)$ karmaşıklığına sahip ancak fazladan $O(n)$ bellek alanına ihtiyaç duyuyor.
- **Bitonic Sort:** Paralel donanımlar düşünülerek tasarlanmış bir algoritma; birleştirme ağları kullanarak diziyi bitonik sekanslara dönüştürüp sıralıyor. Seri çalışma için $O(n \log^2 n)$ karmaşıklığı verdiğinden, tek çekirdekte çalışırken diğer logaritmik algoritmalara göre

işlemciye daha fazla yük çıkarıyor.

Seri Çalışma Sonuçlarım:

(Not: $O(n^2)$ karmaşıklığa sahip Bubble ve Selection Sort algoritmalarını 100.000 eleman üzerinden ölçüp, sonuçları 1.000.000 elemana ekstrapolasyon yöntemiyle genelledim.)

- **Quick Sort:** 0.12s — $O(n \log n)$
- **Merge Sort:** 0.16s — $O(n \log n)$
- **Bitonic Sort:** 0.39s — $O(n \log^2 n)$
- **Selection Sort:** ~1,828.52s — $O(n^2)$
- **Bubble Sort:** ~3,863.56s — $O(n^2)$

Veriler, teorik Big-O notasyonlarının gerçek hayattaki karşılıklarını birebir doğruladı. Beklediğim gibi Quick Sort (0.12s) seri çalışmada en hızlı algoritma oldu. Bellek üzerinde yeni dizi tahsis etmemesi ve işlemcinin L1/L2 önbelleğini verimli kullanması sayesinde, teoride aynı karmaşıklığa sahip olduğu Merge Sort'a kıyasla yaklaşık 2.8x daha hızlı çalıştı. Bitonic Sort (0.39s), paralel donanımlara uygun tasarlandığından seri ortamda dezavantajlı kalarak yavaş çalıştı. $O(n^2)$ tabanlı algoritmalar ise (Selection: 30.5 dakika, Bubble: 64 dakika) bu kadar büyük veri setleri için kesinlikle kullanışsız.

3. Paralel (MPI) Algoritmalar ve Hızlanma (Speedup) Analizi

Paralel versiyonları yazarken OpenMPI kütüphanesini kullandım. Süre ölçümlerini `MPI_Wtime()` ile yaptım ve her ölçümde işi en son bitiren çekirdeğin süresini (`MPI_Reduce(MPI_MAX)`) baz aldım. Paralel yürütme akışı şu şekilde gerçekleşiyor: Master çekirdek (Rank 0) veriyi diğerlerine `MPI_Scatter` fonksiyonu ile dağıtıyor. Her çekirdek kendi local verisi üzerinde hesaplama yapıyor. Ardından `MPI_Sendrecv` kullanılarak karşılaştırma ve birleştirme operasyonları yapılıyor. Finalde sonuçlar Rank 0 üzerinde `MPI_Gather` ile toplanıyor.

8 Çekirdekte Elde Ettiğim Hızlanmalar ($S = T_{seri} / T_{paralel}$):

- **Quick Sort:** 4.44x
- **Merge Sort:** 5.65x
- **Bitonic Sort:** 6.29x

Paralel sonuçları analiz ettiğimde hiçbir algoritmanın lineer ($S = p$) bir hızlanma elde edemediğini gözlemledim. Bunun başlıca temel nedenleri şunlar:

1. **Amdahl Yasası:** Hiçbir paralel program %100 oranında paralelleştirilemez. Quick Sort uygulamasında pivotun seçilip diğer çekirdeklere yayınlanması seri bir işlem olduğu için 8 çekirdekteki teorik maksimum hızlanmayı sınırladı.
2. **Haberleşme Gecikmesi (Communication Overhead):** MPI mesajlaşma altyapısında her bir paketin transferi gecikme yaratıyor. Çekirdek sayısı arttıkça, birleştirme aşamasında ağ üzerinden aktarılan veri miktarı katlanarak artış gösteriyor.
3. **Yük Dengelemesi (Load Imbalance):** Quick Sort'ta seçilen pivot veriyi her zaman tam ortadan ikiye bölmüyor. Bu durum, bazı çekirdeklerin diğerlerinden daha fazla eleman sıralamasına ve senkronizasyon sürecinde bekleme durumlarına yol açtı. Ancak Bitonic Sort'un

deterministik doğası kusursuz bir yük dengelemesi sağladı ve 8 çekirdekte 6.29x hıza ulaşmasını sağladı. Ayrıca Merge Sort da *divide-and-conquer* yapısıyla iyi bir paralel potansiyel (5.65x) sunarken, Quick Sort veri-bağımlı pivot seçimi ve yük dengesizliği yüzünden daha sınırlı ölçeklenme (4.44x) gösterdi.

4. Kod Mimarisi ve Uygulama Detayları

Kod projesini yazarken aşağıdaki modüler dizin yapısını organize ettim:

- `src/serial/` — C++ ile yazılmış seri algoritmalar (`gettimeofday` ile ölçümlenmiş)
- `src/parallel/` — C ve MPI ile yazılmış paralel algoritmalar (`MPI_Wtime` ile ölçümlenmiş)
- `Makefile` — Otomatik kaynak keşfi ve derleme için
- `reports/` — Zaman damgalı çıktı raporları

Seri versiyon (`main.cpp`) komut satırı argümanını parse ettikten sonra (varsayılan: `all`), rastgele sayı üreticimi 42 seed'i ile başlatıyor, 1 milyon elemanlı dinamik dizi tahsis ediyor ve her algoritma için zaman ölçümü, çalıştırma, doğrulama ve rapor yazma adımlarını izliyor. Paralel versiyon (`main.c`) ise `MPI_Init` ile ortam başlatıldıktan sonra, Rank 0 (master) veriyi üretip `MPI_Scatter` ile dağıtıyor. Tüm ranklar kendi local verisi üzerinde hesaplamasını yapıyor, `MPI_Sendrecv` ile compare-exchange operasyonları yürütülüyor ve Rank 0 `MPI_Gather` ile işi toparlayıp raporluyor.